

APPUNTI COMPLETI COMPLEMENTI DI RETI

Il livello rete è nato per risolvere il problema di interconnettere dispositivi provenienti da diverse reti locali senza dover creare un'unica gigantesca rete fisica basata sulla stessa tecnologia.

La soluzione è stata creare un livello software sovrapposto con propri sistemi intermedi (i router) e un proprio sistema di indirizzamento (gli indirizzi IP)

Per far funzionare questo livello sono necessari 2 processi:

- **Forwarding** (inoltro): processo **locale** che avviene a livello del singolo router. Il router analizza **l'intestazione del pacchetto IP** in arrivo e, consultando la **forwarding table**, determina il collegamento in uscita sul quale inviarlo.
- **Routing** (instradamento): processo **distribuito** a livello di intera rete che serve a **calcolare i percorsi migliori** e a popolare automaticamente le forwarding table di tutti i router.

ALGORITMI E METRICHE DI ROUTING

Per determinare quale sia il percorso migliore tra due punti, gli algoritmi utilizzano due diverse categorie di metriche:

- **Metriche statiche:** non dipendono dallo stato attuale della rete, includono il numero di nodi attraversati, i costi amministrativi o il ritardo statico.
- **Metriche dinamiche:** variano in base allo stato in tempo reale della rete, considerando fattori come il carico del collegamento o il ritardo attuale dei pacchetti.

Esistono diverse famiglie di protocolli di routing:

- **Distance Vector:** i router conoscono solo il proprio stato locale e stimano i percorsi basandosi sulle informazioni ricevute dai vicini tramite l'algoritmo di Bellman-Ford. (ad esempio i protocolli RIP e IGRP)
- **Link State** (stato del collegamento): ogni router costruisce una mappa completa (topologia) della rete basata sulla conoscenza globale dello stato dei collegamenti. Calcola poi il percorso ottimale utilizzando l'algoritmo di Dijkstra (shortest path first). OSPF è il protocollo Link State più famoso.
- **Path Vector:** non richiede conoscenza globale, ma si basa su una sequenza di domini amministrativi attraversati, rendendolo ideale per l'instradamento su scala globale. BGP, il protocollo che tiene in piedi l'Internet moderno, appartiene a questa categoria.
- **Flooding:** un metodo basilare in cui ogni pacchetto viene duplicato su ogni collegamento in uscita. È un sistema molto robusto ma estremamente costoso in termini di traffico generato.

DIFFERENZA TRA HOST E ROUTER

- I normali host hanno uno stack di rete ed eseguono un routing elementare.
- Tramite una tabella di routing interna, un PC deve decidere se inviare un pacchetto alla propria scheda di rete fisica (NIC) per farlo uscire, oppure all'interfaccia virtuale di loopback se il servizio è ospitato sulla macchina stessa.
- Un router (o gateway) si distingue perché possiede più di una interfaccia di rete e ha il compito specifico di ricevere pacchetti non destinati a lui per poi re-immetterli nel processo di routing. (forwarding)

LA SUITE FRROUTING

FRR è il successore spirituale di progetti storici come Zebra e Quagga, ed è oggi uno standard di fatto per trasformare macchine Linux standard in router estremamente avanzati, capaci di competere con hardware proprietario e costoso.

FRR supporta numerosi protocolli, tra cui RIP, OSPF, IS-IS, BGP.

L'architettura software di FRR è modulare:

- Esistono vari **daemons** (processi in background) specifici per ogni protocollo, come `ripd`, `ospfd` e `bgpd`
- Tutti questi daemons comunicano con un daemon centrale chiamato **zebra**.
- Il daemon zebra funge da intermediario esclusivo per comunicare le rotte calcolate al kernel del sistema operativo

CONFIGURAZIONE E GESTIONE TRAMITE VTYSH

Invece di configurare ogni daemon separatamente, FRR offre una **shell unificata** chiamata **vtysh**. Il file di configurazione `/etc/frr/daemons` permette di scegliere quali demoni avviare all'accensione.

La gestione tramite vtysh prevede un sistema di privilegi a livelli di sicurezza:

- **Utente non privilegiato** (Default): può visualizzare solo informazioni non critiche.
- **Utente privilegiato**: vi si accede con il comando `enable` e permette di visualizzare informazioni critiche e fare configurazioni elementari.
- **Configuratore**: vi si accede con il comando `configure` e permette di applicare configurazioni persistenti che alterano il comportamento del router.

La configurazione di FRR è definita "operazionale" e non dichiarativa: per rimuovere un'impostazione precedentemente inserita, è necessario digitare lo stesso comando preceduto dalla particella "no".

Comandi principali disponibili nella shell vtysh

Comando	Descrizione
<code>?</code>	Mostra l'elenco dei comandi disponibili
<code>configure</code>	Entra nella modalità di configurazione
<code>enable</code>	Attiva la modalità utente privilegiata
<code>exit / quit</code>	Esce dalla modalità attuale e torna a quella precedente
<code>no</code>	Nega un comando o ripristina i suoi valori predefiniti
<code>ping</code>	Invia messaggi di echo per testare la connettività
<code>show</code>	Mostra le informazioni sul sistema in esecuzione (es. <code>show ip route</code>)
<code>terminal</code>	Imposta i parametri della linea del terminale
<code>write</code>	Salva la configurazione corrente in memoria o sul file system

IPERF3

Iperf3 è uno strumento attivo che genera del traffico sintetico in condizioni controllate per testare i limiti fisici e logici della rete.

Funziona con un modello client-server

ARCHITETTURA DI BASE (Client - Server)

Occorre l'utilizzo di due macchine:

- **Server:** si mette in ascolto (su porta TCP 5201) in attesa di connessioni utilizzando il comando `iperf3 -s`. Può essere eseguito anche in background (tramite daemon) con l'opzione `-D`.
- **Client:** inizia la misurazione connettendosi al server, generando il traffico e calcolando le statistiche finali. Il comando è `iperf3 -c <IP_SERVER>`

MISURAZIONE DELLE PRESTAZIONI TCP

Il protocollo TCP garantisce la consegna dei pacchetti, ma è soggetto a meccanismi di controllo che possono limitarne la velocità:

- **Flussi paralleli (-P):** un singolo flusso TCP spesso non riesce a saturare un collegamento ad altissima velocità a causa dei limiti della congestion window (quantità di dati che può essere inviata prima di attendere una conferma) .
Usando il flag `-P 4` si aprono 4 connessioni simultanee per massimizzare l'uso della banda.
- **TCP Window Size e BDP (-w):** su reti con alta latenza, la dimensione del buffer TCP può fare da collo di bottiglia. Questo è legato al concetto di **Bandwidth-Delay Product (BDP)**, ovvero la quantità di dati "in volo" sulla rete in un dato momento.
La formula teorica è $\text{Buffer} = \text{Bandwidth} * \text{RTT}$.
`iperf3` permette di forzare questa dimensione, ad esempio con `-w 256K`
- **Algoritmi di congestione (-C):** il TCP decide quanto velocemente inviare i dati basandosi su algoritmi specifici. `Iperf3` permette di testare e confrontare algoritmi storici come RENO (basato su perdita di pacchetti), lo standard Linux CUBIC, o il più moderno BBR di Google (che cerca di massimizzare la banda e minimizzare la latenza)

MISURAZIONE DELLE PRESTAZIONI UDP E QoS

L'UDP non garantisce la consegna dei pacchetti, ma è velocissimo ed è il fondamento delle applicazioni in tempo reale (VoIP, streaming video, gaming).

In questa modalità (`-u`), `iperf3` richiede che tu specifichi la larghezza di banda target usando il flag `-b`.

I parametri critici per il real time:

- **Packet Loss:** quanti pacchetti non sono mai arrivati a destinazione
- **Jitter:** la variazione del ritardo tra i pacchetti in arrivo. Se il valore è troppo alto (superiore a 30ms), l'audio di una chiamata VoIP arriverà a scatti o in disordine

VALIDAZIONE AVANZATA CON LINUX TRAFFIC CONTROL (tc)

`Iperf3` può essere utilizzato in combinazione con il comando `tc` (Traffic Control) per simulare reti imperfette o validare policy di rete.

- **TBF** (Token Bucket Filter): utilizzato per limitare artificialmente la banda in uscita. Puoi impostare un limite a 10MB/s tramite `tc` e verificare con `iperf3` se il limite viene rispettato.
- **Netem** (Network Emulator): permette di iniettare deliberatamente delay, jitter e perdita di pacchetti nell'interfaccia di rete del server. Utile per vedere come si comporterebbe un'applicazione se la connessione del server fosse scarsa.

- **HTB** (Hierarchial Token Bucket): serve a creare “corsie preferenziali” per certo tipo di traffico.

AUTOMAZIONE E SCRIPTING

Iperf3 permette di restituire il risultato in JSON tramite il flag `-J`.

- Ciò permette di integrare facilmente iperf3 all'interno di script (es. Python) per automatizzare il monitoraggio della rete.
- Estrarre chiavi JSON come `.end.sum_received` o `.end.streams[n].udp.jitter_ms` permette di creare grafici automatizzati dell'andamento della rete nel tempo.

RIP – ALGORITMO DISTANCE VECTOR

RIP si basa fondamentalmente sull'algoritmo di routing Distance-Vector (Bellman-Ford)

- **Come funziona:** i router apprendono la topologia di rete “scambiandosi informazioni” con i router vicini.
In ogni ciclo, un router utilizza la propria tabella di routing (i percorsi a costo minimo) e le tabelle ricevute dai router vicini per calcolare i percorsi migliori verso tutte le destinazioni conosciute.
- **Terminazione:** il processo si ripete finchè la **rete non converge**, ovvero finchè nessun router ha nuovi aggiornamenti da condividere (l'input corrisponde all'output)

IL PROBLEMA COUNT-TO-INFINITY

Un errore nelle implementazioni ingenuie del Distance Vector è il **Routing Loop**:

- Se un collegamento si interrompe improvvisamente (ad es. Il router Y perde la connessione diretta con il router X), il router Y potrebbe erroneamente pensare di poter raggiungere X instradando il traffico attraverso il router Z.
- Tuttavia, il router Z si affidava al router Y per raggiungere X.
- Di conseguenza, il traffico viene continuamente rimbalzato avanti e indietro, incrementando costantemente il costo per raggiungere X. Questo ciclo richiede un tempo di risoluzione molto lungo, da cui il termine **“count to infinity”**.

SOLUZIONE – POISONED REVERSE

Per prevenire i routing loops, RIP utilizza un meccanismo chiamato **Poisoned Reverse**.

- **La regola:** se il router Z utilizza il router Y per raggiungere una destinazione (X), il router Z comunicherà attivamente al router Y che la sua distanza da X è infinita
- **Il risultato:** poichè Y ritiene che Z non abbia un percorso valido per raggiungere X, Y non tenterà mai di instradare il traffico per X nuovamente attraverso Z se il collegamento diretto fallisce. Ciò costringe l'algoritmo a riconvergere rapidamente.

4. Implementazione nel Laboratorio Kathará (FRR)

La seconda parte del documento fornisce una guida pratica per configurare RIPv2 utilizzando la suite **FRRouting (FRR)** all'interno dell'emulatore Kathará.

- **Avvio dei Demoni:** A differenza dei normali servizi di sistema, FRR potrebbe non essere in esecuzione di default. Deve essere avviato manualmente su ciascun router utilizzando il comando `systemctl start frr`.
- **Configurazione tramite vtysh:** RIP viene configurato entrando nella shell unificata `vysh`. Da qui, si abilita il protocollo e si dichiarano le reti locali che il router deve annunciare agli altri (es. tramite il comando `network 100.1.0.0/16`).
- **Comunicazione Multicast:** Nelle versioni moderne (RIPv2), gli aggiornamenti di routing non vengono inviati in broadcast a tutti, ma inviati in modo più efficiente tramite pacchetti multicast all'indirizzo IP dedicato **224.0.0.9**.
- **Routing Statico e Internet:** Il laboratorio mostra uno scenario in cui la rete locale (una "stub network") ha un solo punto di uscita verso Internet (es. il Router R5). In questo caso, basta configurare una rotta statica predefinita (`ip route add default via...`). È poi possibile "iniettare" questa rotta in tutto il dominio RIP usando il comando `route 0.0.0.0/0` dentro la configurazione di FRR, in modo che tutti i router interni sappiano come uscire su Internet.
- **Redistribute Connected:** Il documento precisa che, per impostazione predefinita, RIP propaga solo le subnet connesse alle interfacce in cui RIP è esplicitamente abilitato. Il comando `redistribute connected` forza invece il router a condividere le informazioni di tutte le sue reti direttamente connesse, comprese quelle dove RIP non "parla".

PASSAGGIO DA IPv4 A IPv6

Il protocollo IPv4 offre circa 4,3 miliardi di indirizzi.

IPv6 è stato progettato per risolvere il problema alla radice:

- **Spazio di indirizzamento enorme:** passa da 32bit di IPv4 ai 128bit. Ciò significa avere a disposizione $3.4 \cdot 10^{38}$ indirizzi.
- **Niente più NAT:** il NAT viene usato per far condividere un solo IP pubblico a una intera rete privata, non è più necessario.

STRUTTURA E RAPPRESENTAZIONE DELL'INDIRIZZO

Essendo lunghi 128 bit, gli indirizzi IPv6 non si scrivono più in **formato decimale separato da punti**, ma in formato esadecimale separato da “:”, diviso in 8 gruppi da 4 cifre esadecimali.

Regole di compressione:

- Si possono omettere gli zeri iniziali di ogni gruppo (es. 0db8 diventa db8)
- Una sequenza continua di gruppi composti solo da zeri può essere sostituita, **una sola volta** in tutto l'indirizzo, dal simbolo ::

Es. 2001:0db8:0000:0000:0000:ff00:0042:8329 diventa

2001:db8::ff00:42:8329

I TIPI DI INDIRIZZI IPv6

IPv6 elimina completamente il concetto di indirizzo Broadcast (che in IPv4 causava rumore sulla rete), sostituendolo con meccanismi più efficienti. Esistono tre tipologie di indirizzi:

- **Unicast:** identifica una singola interfaccia
 - **Global Unicast:** equivalente agli indirizzi pubblici IPv4, instradabili su Internet
 - **Link-Local:** ogni interfaccia IPv6 genera automaticamente un indirizzo Link-Local (inizia sempre con fe80::). Questo indirizzo è valido solo all'interno di quel segmento di rete (non attraversa i router) ed è fondamentale per protocolli di routing (es. OSPFv3) e per la scoperta dei vicini
- **Multicast:** un pacchetto inviato a un indirizzo multicast viene consegnato a tutte le interfacce che si sono unite a quel gruppo multicast
- **Anycast:** un indirizzo assegnato a più dispositivi (spesso server). Il traffico viene consegnato solo al dispositivo più vicino (secondo le metriche di routing)

CONFIGURAZIONE PRATICA – Katharà

La configurazione di IPv6 sfrutta i comandi standard della suite **iproute2** di Linux ma con il **flag “-6”**.

- **Assegnare un indirizzo IPv6:** *ip -6 addr add 2001:db8:1::1/64 dev eth0*
- **Vedere gli indirizzi configurati:** *ip -6 addr show*
- **Aggiungere una rotta statica predefinita:**
ip -6 route add default via 2001:db8:1::2
- **Testare la connettività:**
ping6 2001:db8:1::2 OPPURE ping -6

NEIGHBOR DISCOVERY PROTOCOL – NDP

In IPv4, quando un PC doveva scoprire il MAC address del vicino, comunicava in broadcast tramite protocollo ARP.

In IPv6 l'ARP non esiste più: è stato sostituito dal NDP, che sfrutta speciali messaggi ICMPv6 di tipo Multicast.

NDP è molto più efficiente e non disturba chi non è interessato alla comunicazione, gestisce anche la Stateless Address Autoconfiguration (SLAAC), che permette ai dispositivi di autoconfigurarsi un IP pubblico semplicemente parlando con il router locale.

ADDRESS RESOLUTION PROTOCOL – ARP

Per capire l'ARP dobbiamo pensare a come comunicano i computer su una rete.

- **A livello Rete** i computer si identificano **tramite indirizzi IP** (indirizzo logico)
- Per far viaggiare fisicamente i dati sul cavo (o WiFi) a **livello Collegamento**, le schede di rete utilizzano gli **indirizzi MAC** (indirizzo fisico)

Problema: Se il PC-A vuole inviare un pacchetto IP al PC-B che si trova sulla sua stessa rete locale, conosce il suo indirizzo IP, ma per incapsulare il pacchetto in un frame Ethernet ha bisogno del MAC address del PC-B, come scoprirlo?

Soluzione: Utilizzo del protocollo ARP, che funge da “elenco telefonico” dinamico, traducendo gli indirizzi IP in indirizzi MAC

COME FUNZIONA ARP

Il protocollo ARP funziona in due fasi (si possono identificare tramite comando tcpdump):

- **ARP Request:** Il PC-A invia un pacchetto a tutti i dispositivi sulla rete locale (utilizzando il MAC di broadcast). Sostanzialmente chiede “Chi ha l'indirizzo IP x.x.x.x? Restituisca il MAC al MAC-A
- **ARP Reply:** Tutti i dispositivi ricevono la richiesta ma solo il PC-B risponde inviando un pacchetto Unicast dicendo “Io sono x.x.x.x e questo è il mio MAC : MAC-B”

Ora il PC-A può costruire il frame Ethernet e inviare i dati

LA CACHE ARP

Sarebbe molto inefficiente se i computer dovessero inviare una ARP Request per ogni singolo pacchetto di dati.

Per evitare ciò, i sistemi operativi (compresi host e router) mantengono una cache ARP (**tabella ARP**), ovvero una memoria temporanea che salva le associazioni IP-MAC apprese di recente. La tabella scade dopo un certo timeout (periodo di inattività), ciò garantisce che se un dispositivo dovesse cambiare scheda di rete, la rete possa accorgersene.

Per ispezionare la tabella ARP si possono sfruttare due comandi:

- *arp -n*
- *ip neigh show* (comando della suite iproute2)

VEDERE IL TRAFFICO ARP

Per vedere il traffico ARP si possono utilizzare i seguenti comandi:

1. **Pulire la cache:** spesso si forza la pulizia della Tabella ARP per forzare il sistema a generare nuove richieste
ip neigh flush all
2. **Avviare lo sniffing:** si esegue un comando per mettere il sistema nelle condizioni di ascoltare solo il traffico ARP
tcpdump -ttni eth0 arp
3. **Generare traffico:** si lancia il comando ping verso un altro host
4. **Osservazione:** sul terminale di tcpdump si vedrà l'intera transazione: prima la Request in broadcast, poi la Reply in Unicast e subito dopo i pacchetti ICMP del ping che iniziano a fluire

ALTRI UTILIZZI DI ARP

- **Gratuitous ARP** (ARP gratuito): è una **ARP Reply (o Request) inviata da un host senza che nessuno glielo abbia chiesto**. Viene usata principalmente quando un computer si accende o cambia IP, per avvisare preventivamente a tutti gli switch e gli altri host la nuova associazione IP-MAC.
Fondamentale per rilevare conflitti di IP (due dispositivi con lo stesso indirizzo)
- **Proxy ARP:** avviene quando un router risponde a una ARP Request per conto di un altro dispositivo che si trova su rete diversa. **Il router finge di essere la destinazione finale (dando il proprio indirizzo MAC) per attrarre il traffico e poi instradarlo**. Spesso è disabilitato per motivi di sicurezza, ma storicamente è stato molto utile in diverse topologie.

OSPF – Open Shortest Path First

OSPF è un **protocollo di routing interno** (Interior Gateway Protocol) utilizzato nelle reti di grandi dimensioni come quelle degli ISP.

Mentre protocolli come RIP utilizzano la logica Distance-Vector (i router conoscono la rete per “sentito dire” dai propri vicini), OSPF utilizza un approccio Link-State.

- **La mappa della Rete:** ogni router invia aggiornamenti sullo stato dei propri collegamenti (Link State Updates) tramite un **meccanismo di inondazione (flooding) a tutti i router della Rete**
- **Il database (LSDB):** utilizzando questi aggiornamenti, ogni router costruisce un **Link State Database (LSDB) identico a quello di tutti gli altri**. Questa è la mappa topologica completa della rete.
- **L'algoritmo:** su questo database, **ogni router esegue in autonomia l'algoritmo di Dijkstra per calcolare l'albero dei percorsi a costo minimo (Shortest Path Tree) verso ogni destinazione**, ponendo se stesso alla radice dell'albero.
- Questo approccio **elimina i problemi di “routing loop”** tipici del Distance-Vector e permette di gestire reti illimitate in termini di salti (risulta più complesso dal punto di vista computazionale)

TIPI DI MESSAGGI OSPF

Per far funzionare questo sistema, OSPF usa 5 tipi di messaggi:

- **Hello:** per scoprire e mantenere il contatto con i vicini.
- **Database Description (DD / LSDB):** quando due router si incontrano, si scambiano un riassunto dei loro database per capire chi ha le informazioni più aggiornate.
- **Link State Request (LSR):** Usato per richiedere i dettagli di specifici LSA mancanti.
- **Link State Update (LSU):** Fornisce i dati effettivi richiesti o comunica un cambiamento di stato.
- **Link State Ack (LSAck):** Conferma la ricezione di un aggiornamento per garantire l'affidabilità.

OSPF funziona direttamente sopra il livello IP e usa specifici indirizzi Multicast (224.0.0.5 per tutti i router e 224.0.0.6 per i router designati) per limitare il traffico broadcast superfluo.

- **Le metriche:** il calcolo dei percorsi migliori non avviene contando i salti, ma valutando i costi dei collegamenti.

Di base, il costo è proporzionale all'inverso della larghezza di banda dell'interfaccia ($\text{Cost} = 10^8 / \text{bandwidth}$), ma può essere alterato artificialmente dagli amministratori per deviare il traffico su strade preferenziali tramite il comando `ospf cost X` (con X sostituito dal valore del costo che si intende assegnare)

I Designated Routers (DR) e Backup (BDR)

Sui collegamenti condivisi come le LAN, c'è un problema di scalabilità: se ci sono n router, si verrebbero a creare $n(n-1) / 2$ connessioni tra di loro, sommergendo la rete di traffico LSA per sincronizzarsi.

Per risolvere il problema, su ogni rete multi-accesso viene eletto un Designated Router (DR) e un suo sostituto, il Backup Designated Router (BDR).

Gli altri router stabiliscono una adiacenza "Full" (cioè si scambiano il database solo con DR e BDR)

L'elezione del DR avviene tramite i pacchetti Hello e si basa sulla priorità dell'interfaccia (da 0 a 255, dove 0 significa che non diventerà mai DR).

In caso di parità, vince il router con l'indirizzo IP (Router ID) più alto.

Cambiare un DR equivale a cambiare la topologia per OSPF, per questo il sistema è conservativo: un nuovo router con priorità più alta NON ruba il posto a un DR già esistente.

IL ROUTING GERARCHICO: Le aree OSPF

L'algoritmo di Dijkstra, se eseguito su una rete gigantesca, consumerebbe troppa CPU e memoria (la sua complessità cresce al crescere di nodi e link). OSPF risolve dividendo la rete in blocchi chiamati **Aree**:

- **Area 0 (backbone)**: è il cuore pulsante. Tutti i percorsi tra aree diverse (Inter-area) devono fisicamente e logicamente attraversare il Backbone.
I router assumono ruoli specifici:
 - **Internal Routers**: tutte le loro interfacce sono in una sola area; conoscono solo i dettagli della propria area.
 - **Area Border Routers (ABR)**: si trovano sul confine tra l'area 0 e le altre aree.
Mantengono un LSDB separato per ciascuna area e scambiano pacchetti riassuntivi tra di esse.
 - **Autonomous System Boundary Routers (ASBR)**: iniettano in OSPF le rotte imparate da altri domini e altri protocolli, come il BGP.
- **Aree stub**: per risparmiare ulteriore memoria, alcune aree finali della rete vengono configurate come "Stub". Questo significa che l'ABR filtra tutte le pesanti rotte esterne (quelle di Internet imparate via BGP) e fornisce ai router vicini una singola rotta predefinita (Default Route) per uscire.

TIPI E PRIORITA' DEI PERCORSI (Path Types)

In OSPF, non tutti i percorsi hanno lo stesso peso topologico. Vengono scelti con questo ordine di preferenza assoluto:

- **Intra-Area**: un percorso interno alla propria area.
- **Inter-Area**: un percorso che attraversa un ABR verso il Backbone e poi scende verso un'altra area.
- **External Type 1 (E1)**: rotte importate da protocolli esterni. Il costo è la somma logica del costo esterno + il costo interno di OSPF per arrivare all'ASBR.
- **External Type 2 (E2)**: rotte importate in cui si calcola solo il costo metrico del protocollo esterno. La distanza in OSPF verso l'ASBR viene usata solo in caso di parità tra due percorsi esterni identici.

TEMPI DI CONVERGENZA (Reazione ai guasti)

OSPF è molto veloce nel reagire ai cambiamenti:

- **Link Fault** (guasto di un cavo): se l'interfaccia cade fisicamente, la propagazione del messaggio LSA per rimuovere quel collegamento dal database è immediata e la rete si riconfigura istantaneamente.
- **Router Fault** (guasto hardware improvviso): se un intero router si spegne o va in crash senza avvisare, i vicini se ne accorgeranno solo allo scadere del RouterDeadInterval (timer impostato di base a 40 secondi, che equivalgono a 4 messaggi Hello non ricevuti). Solo a quel punto aggiorneranno le tabelle ed eleggeranno un nuovo DR (se necessario).
Le informazioni vecchie persistono nei database altrui finché non scadono naturalmente (in genere il MaxAge è di 1 ora), ma vengono bypassate se il DR ricostruisce la mappa senza di esse.

ANATOMIA DEL LINK STATE DATABASE (LSDB) E TIPI DI LSA

Il database è composto da diversi tipi di record LSA (Link State Advertisement) che descrivono entità diverse:

- **Router LSA** (tipo 1): prodotti da ogni router per descrivere lo stato delle proprie interfacce fisiche. In questo caso, l'identificativo (Link ID) corrisponde al Router ID del router stesso.
Per vederli si usa: *show ip ospf database router*
- **Network LSA** (tipo 2): prodotti esclusivamente dai Designated Router (DR) per rappresentare le reti multi-accesso (es. LAN) e i router a esse collegati. Il Link ID corrisponde all'indirizzo IP del DR.
Si ispezionano con: *show ip ospf database network*
- **Summary LSA**: generati dagli Area Border Router (ABR) per comunicare a un'area i percorsi verso le reti di un'altra area
Visibile tramite: *show ip ospf database summary*
Mostrano a quale distanza metrica si trova la destinazione rispetto all'ABR che la annuncia.
- **ASBR-Summary LSA**: descrivono come raggiungere i router di confine (ASBR) che collegano ad altri domini di routing.

GESTIONE DEI PERCORSI MULTIPLI (Multipath) E FRROUTING (Zebra)

Cosa succede se OSPF trova due strade diverse che hanno esattamente lo stesso costo complessivo per raggiungere una destinazione?

- **OSPF implementa l'Equal Cost MultiPath (ECMP)**, quando ci sono più rotte con costo uguale, OSPF le inserisce tutte nella sua tabella di routing (*show ip route* per vederla).
- In FRROUTING, OSPF delega al demone centrale (**Zebra**) il compito di effettuare la selezione finale e il bilanciamento del carico tra questi percorsi di pari costo prima di inviarli al kernel del sistema operativo.

CONFIGURAZIONE DELLE STUB NETWORK

Per gestire le reti locali terminali (stub network), cioè le reti in cui è presente un solo router OSPF e a cui sono collegati i normali host, ci sono 3 diversi scenari:

- **OSPF abilitato sull'interfaccia**: la rete viene annunciata in tutto il dominio OSPF, ma il router invia inutilmente altri pacchetti Hello sulla LAN cercando vicini che non esistono.
- **OSPF non abilitato sull'interfaccia**: i pacchetti Hello non vengono inviati, ma la rete verrà annunciata solo come rotta esterna (AS-external route) tramite redistribuzione, finendo per transitare solo nel Backbone.
- **OSPF abilitato come interfaccia passiva: considerata la pratica migliore**, la rete viene annunciata correttamente in tutto il dominio OSPF interno, ma il router sopprime l'invio dei pacchetti Hello su quella interfaccia, risparmiando banda e risorse.

TIMER E CONVERGENZA SUB-SECONDO

Gli LSA subiscono un refresh ogni 30 minuti, scadono dopo 1 ora (**magAge**).

La morte di un vicino viene rilevata in 40 secondi (**DeadInterval**) e l'esecuzione di un algoritmo SPF è limitata a intervalli di 30 secondi (**SPF Throttle**) per evitare sovraccarichi.

Questi tempi creano un tempo di convergenza totale di circa 40 secondi, considerati eccessivi per le reti moderne ad alte prestazioni.

Le soluzioni per velocizzare includono:

- Ridurre il timer dei pacchetti Hello a 100ms per rilevare i guasti in modo rapido.
- Eseguire il ricalcolo di Dijkstra senza throttling.
- Delegare il monitoraggio al **protocollo BFD (Bidirectional Forwarding Detection)** per abbattere i tempi di convergenza al di sotto del secondo

PRO E CONTRO DI OSPF

- **Vantaggi:** non soffre del problema "count to infinity", non ha limiti operativi sui salti (i limiti teorici si aggirano sui 5000 link, dovuti alla dimensione massima dei pacchetti LSA di 64KB), genera molto meno traffico di segnalazione (aggiornamenti ogni 30 minuti invece che comunicazioni in broadcast, come nel RIP) e utilizzo di metriche avanzate.
- **Svantaggi:** protocollo molto complesso, richiede una progettazione della rete molto precisa ed è molto più difficile da configurare e gestire rispetto alla semplicità del RIP.

TRAFFIC CONTROL DI LINUX

Per usare tc bisogna comprendere la sua architettura, basata su **3 pilastri**:

- **qdisc** (Queueing Discipline): elemento fondamentale, decide come i pacchetti devono essere messi in coda e spediti fuori da una interfaccia di rete.
- **Class:** alcune qdisc (chiamate classful) possono essere suddivise in "classi" gerarchiche, permettendo di assegnare limiti di banda diversi a code diverse.
- **Filter:** è la regola che intercetta il pacchetto in base a criteri specifici (indirizzo IP, porta, protocollo) e lo assegna alla classe corretta.

Le principali qdisc:

- **pfifo / bfifo:** la coda più elementare. Il primo pacchetto che entra è il primo a uscire. Si può limitare la coda per numero di pacchetti (pfifo limit 1000) o per byte (bfifo).
- **TBF** (Token Bucket Filter): strumento per imporre un limite rigido alla banda in uscita, funziona come un secchio in cui cadono dei "gettoni" a una velocità costante. Un pacchetto può uscire solo se prende un gettone. Ideale per "strozzare" l'interfaccia (es. bloccare eth0 a 1Mbit/s) senza causare accumuli eccessivi nel buffer.
- **HTB** (Hierarchical Token Buffer): qdisc classful più usata. Permette di creare una struttura ad albero: ad esempio, su una banda totale di 100 Mbit/s, puoi garantire 10 Mbit/s al traffico ad alta priorità (VoIP) e 5 Mbit/s al traffico generico. Tramite il parametro ceil se il traffico generato dal canale con banda maggiore non la riempie allora gli altri canali possono utilizzare la banda in eccesso fino a che non servirà al canale dedicato.

- **fq_codel** (Fair Queuing + Controlled Delay): perfetta per risolvere il bufferbloat (accumulo di latenza causato da buffer troppo grandi nei router moderni). Attualmente è la qdisc consigliata di default su Linux, garantisce bassa latenza a tutti.
- **netem** (Network Emulator): serve per peggiorare (intenzionalmente) la rete a scopo di test. Permette di iniettare delay, variazioni di ritardo (jitter), packetloss o corruzione. esempio di utilizzo: `tc qdisc add dev eth0 root netem delay 100ms loss 1%`

GESTIRE IL TRAFFICO IN ENTRATA

Solitamente tc viene utilizzato per gestire il traffico in uscita ma può anche limitare la banda in ingresso.

Esiste una qdisc chiamata ingress abbinata a una azione di police. Questo metodo intercetta il traffico in arrivo e semplicemente “butta via” i pacchetti che superano il limite imposto, costringendo il protocollo TCP del mittente a rallentare.

AUTOMAZIONE E TRAPPOLE

- **Volatilità:** le regole impostate con tc spariscono al riavvio del server. Vanno scriptate in /etc/rc.local o gestite tramite wrapper scritti in Python come tcconfig.
- **Pulizia:** prima di applicare nuove regole complesse, è buona norma cancellare la coda principale con `tc qdisc del dev eth0 root` per evitare sovrapposizioni e conflitti.

DATAPLANE vs CONTROL PLANE

Il livello rete svolge due funzioni fondamentali:

- **Inoltro** (forwarding – Data Plane)
- **Instradamento** (routing – Control Plane)

Il Control Plane può essere implementato in due modi:

- **Per-Router** (tradizionale): ogni router ha il suo algoritmo di routing integrato che comunica con gli altri router per calcolare le tabelle in modo distribuito.
- **Logicamente centralizzato** (SDN): un controller remoto calcola i percorsi e installa direttamente le tabelle di inoltro degli switch.

INOLTRO DIRETTO, INDIRETTO E LONGEST PREFIX MATCH

I router e gli host usano la Subnet Mask per capire come inoltrare i pacchetti:

- **Inoltro diretto:** se l'indirizzo IP di destinazione e quello sorgente, messi in **AND logico con la netmask**, appartengono alla stessa sottorete, il pacchetto viene consegnato direttamente tramite l'indirizzo fisico (MAC).
- **Inoltro indiretto:** **se le sottoreti non corrispondono**, il pacchetto deve essere inviato al router di default (next-hop).

Quando un router legge la sua tabella per inoltrare un pacchetto e **trova più corrispondenze valide (CIDR / Classless addressing)**, usa la regola del **Longest Prefix Match (LPM): vince la rotta con la maschera di sottorete più specifica (cioè con il numero maggiore di bit '1')**. Questo meccanismo permette l'aggregazione delle rotte, riducendo le dimensioni delle tabelle di routing.

ALGORITMI DI ROUTING TRADIZIONALI

Gli algoritmi hanno l'obiettivo di trovare il percorso meno costoso tra sorgente e destinazione. Esistono due grandi famiglie di algoritmi:

- **Link-State / Algoritmo di Dijkstra:**
 - Algoritmo globale: **tutti i nodi ricevono informazioni sull'intera topologia** della rete e sui costi di tutti i link tramite broadcast.
 - Ogni nodo esegue iterativamente l'algoritmo di Dijkstra per calcolare l'albero dei percorsi minimi verso tutte le destinazioni.
 - Complessità di $O(n^2)$, richiede di scambiare messaggi in tutta la rete ($O(n^2)$) ma garantisce calcoli coerenti ed evita loop.
- **Distance-Vector / Algoritmo di Bellman-Ford:**
 - Algoritmo decentralizzato e iterativo: **ogni nodo conosce solo il costo verso i suoi vicini diretti**.
 - Periodicamente, **un nodo invia le sue stime di distanza ai vicini**. I vicini le usano per aggiornare le proprie stime tramite l'equazione Bellman-Ford.
 - Soffre del problema count-to-infinity: le "cattive notizie" impiegano molto tempo a propagarsi, **rischiando di creare loop di routing temporanei**.

PROTOCOLLI DI ROUTING SU LARGA SCALA (Internet)

Internet è divisa in Sistemi Autonomi (AS), ovvero reti gestite da un'unica entità amministrativa (es. ISP), ciò porta a due livelli di routing:

- **Routing Intra-AS** (Interior Gateway Protocol – IGP):
 - **RIP** (Routing Information Protocol): protocollo basato sul Distance Vector, usa come **metrica il conteggio dei salti** con un massimo di 15. Invia aggiornamenti tramite UDP ogni 30 secondi. A causa del count-to-infinity e del limite di grandezza è usato raramente.
 - **OSPF** (Open Shortest Path First): protocollo **Intra-AS più diffuso**. Basato su **Link-State e sull'algoritmo di Dijkstra** (instradamento gerarchico). **Può dividere un AS in aree locali collegate a una Backbone centrale rendendolo estremamente scalabile e veloce nella convergenza**. I messaggi OSPF sono autenticati per prevenire intrusioni.
- **Routing Inter-AS** (Exterior Gateway Protocol – EGP):
 - **BGP** (Border Gateway Protocol): **collante di Internet**. Basato su meccanismo **simile al Distance-Vector chiamato Path Vector: annuncia non solo la distanza, ma l'intero percorso (AS-PATH) per prevenire i loop**. Le decisioni nel BGP non si basano solo sulla performance, ma pesantemente sulle policy aziendali. Usa il TCP (porta 179) e si divide in:
 - **eBGP**: comunicazione tra AS diversi.
 - **iBGP**: distribuzione delle rotte BGP all'interno dello stesso AS.

SDN – Software Defined Networking

Nei router classici (hardware e software chiusi perchè proprietari), il controllo è difficile e le innovazioni sono lente.

Con SDN avviene una rivoluzione simile a quella tra Mainframe e PC:

- Gli switch diventano macchine hardware semplici, veloci e commerciali che si occupano solo del Data Plane.
- Tutta l'intelligenza passa al **Controller SDN** (il **Networking Operating System**), un server centralizzato e distribuito che crea una mappa della rete.
- Sopra al controller ci sono le Applicazioni di Controllo, le API collegano i livelli: **Northbound API** (tra controller e app) e **Southbound API** come OpenFlow (tra controller e switch)
Ciò facilita il Traffic Engineering.

GESTIONE DELLA RETE E CONFIGURAZIONE

Protocolli per monitorare e gestire migliaia di dispositivi complessi:

- **SNMP** (Simple Network Management Protocol): il gestore interroga i dispositivi (request/response) o riceve allarmi automatici (traps).
I dati operativi e di configurazione dei dispositivi sono modellati in alberi strutturati chiamati MIB (Management Information Base).
- **NETCONF e YANG**: rappresentano l'evoluzione della gestione. Invece di configurare un device alla volta, si mira a una configurazione di tutta la rete. YANG è il linguaggio di modellazione dei dati, mentre NETCONF (usa solitamente scambi XML su sessioni sicure) è il protocollo per applicare le configurazioni.

Quando parliamo di OSPF o di altri protocolli interni (IGP), l'obiettivo è semplice: trovare il percorso più veloce (con il costo metrico minore). Quando invece un pacchetto deve viaggiare tra due Sistemi Autonomi (AS) diversi su Internet usando il BGP (Border Gateway Protocol), la logica cambia radicalmente. Nel routing inter-AS, **le policy** (politiche commerciali) **dominano quasi sempre sulle performance**.

No Free Transit

Le reti su Internet stipulano tra loro accordi commerciali, che tipicamente si dividono in due categorie:

- **Provider-Customer** (Fornitore-Cliente): Il cliente paga il provider per accedere al resto di Internet.
- **Peering** (Alla pari): Due reti si scambiano reciprocamente il traffico dei rispettivi clienti senza pagarsi a vicenda (o dividendo i costi), per mutuo vantaggio.

Esempio dello "Stub Customer" (Slide 117):

Immagina una rete aziendale **x** (ad esempio, una grande università o un'azienda) che è "dual-homed", ovvero paga due provider diversi, **B** e **C**, per avere una connessione ridondata. Se il provider **B** vuole mandare traffico al provider **C**, il percorso più breve potrebbe fisicamente passare proprio attraverso la rete del cliente **x**. Tuttavia, l'azienda **x** non vuole assolutamente fare da "passacarte" (router di transito) tra due giganti di Internet, consumando la propria banda che paga a caro prezzo.

La policy BGP: Per evitare questo, il router dell'azienda **x** configurerà il BGP in modo da **non annunciare mai** al provider **B** le rotte che ha imparato dal provider **C** (e viceversa). 🔗 +2

Esempio del Provider "Egoista" (Slide 116):

Immagina ora tre grandi reti di provider: **A**, **B** e **C**. Il provider **A** annuncia le sue rotte a **B** e **C**. Il provider **B** a questo punto conosce la strada per arrivare ad **A**, ma sceglierà deliberatamente di **non annunciare** questa rotta al provider **C**.

Perché? Perché se lo facesse, **C** userebbe la rete di **B** per raggiungere **A**. Dato che né **A** né **C** sono clienti paganti di **B**, il provider **B** non ottiene alcun guadagno (nessuna "revenue") a trasportare questo traffico di transito. Di conseguenza, applicherà una policy BGP per nascondere questa rotta. 🔗 +3

Hot Potato Routing

Supponiamo che un pacchetto entri in un grande Sistema Autonomo (es. AS2) sulla costa est degli Stati Uniti, ma debba raggiungere una destinazione sulla costa ovest (es. AS3). Il router di ingresso (es. 2d) scopre di poter mandare il pacchetto verso la destinazione X usando due gateway di uscita: il router 2a (vicinissimo) o il router 2c (lontano, dall'altra parte del continente).

- Magari, uscendo da 2c, il pacchetto farebbe meno salti totali su Internet per arrivare a destinazione.
- Tuttavia, l'amministratore dell'AS2 configurerà il BGP per usare l'Hot Potato Routing: il router sceglierà sempre il gateway di uscita con il **minor costo interno (intra-domain)**, ignorando totalmente quanto sarà lungo o costoso il viaggio nel resto di Internet.

Come una "patata bollente", la regola dell'ISP è: "Sbarazzati di questo pacchetto il prima possibile passandolo a un'altra rete, così non consuma le mie risorse interne"

COME DECIDE UN ROUTER BGP?

Per applicare tutte queste logiche, quando un router riceve più rotte per la stessa destinazione, esegue un algoritmo di selezione molto rigido, in quest'ordine:

- **Local preferences:** è il valore di policy per eccellenza. Un amministratore lo imposta manualmente per preferire, ad esempio, l'instradamento verso un Customer (da cui guadagna) piuttosto che verso un Provider (a cui deve pagare la banda). Se questo valore è più alto, le altre regole vengono ignorate.
- **Shortest AS-PATH:** a parità di Local Preference, si sceglie il percorso che attraversa il minor numero di Sistemi Autonomi.
- **Closest NEXT-HOP:** a parità dei due valori precedenti, si applica proprio l'Hot Potato Routing, scegliendo il router di uscita fisicamente più vicino.

In sostanza, il BGP è il linguaggio con cui i provider si dicono: "So come arrivare fin lì, ma te lo dico (e ci porto i tuoi pacchetti) solo se mi paghi o se mi conviene".

A differenza degli IGP (es. OSPF, che trova i vicini in broadcast), in BGP i vicini devono essere configurati esplicitamente con l'indirizzo IP.

- BGP utilizza TCP sulla porta 179 per stabilire la sessione.
- Gli annunci contengono il **Prefix** (es. 192.168.1.0/24) e una serie di Attributi del Percorso.

STATI DEL BGP

1. **Idle:** stato iniziale.
2. **Connect:** i router provano ad aprire una sessione TCP.
3. **Active:** in attesa di completare il TCP handshake.
4. **Open Sent / Open Confirm:** i router si scambiano un messaggio Open con i loro parametri (es. numero di AS, BGP Router ID, Hold Time) e rispondono con un Keepalive.
5. **Established:** il peering è stabilito con successo e i router possono scambiarsi i messaggi di Update contenenti le rotte (NLRI).

CONFIGURAZIONE PRATICA – SIMPLE PEERING E ANNUNCI

Come instaurare un eBGP Peering tra AS1 e AS2:

- **Comandi base:** si usa `router bgp 1` per attivare il processo e `neighbor 193.10.11.2 remote-as 2` per dichiarare il vicino.
- **Verifica:** il comando `show ip bgp neighbors` mostra lo stato Established e i capability scambiati.
- **Fare un Annuncio:** per "promettere" al mondo che un AS può consegnare pacchetti verso una specifica destinazione, si usa il comando `network 195.11.14.0/24`.
 - Nota: In FRRouting, per sicurezza, **il router non annuncia nulla via eBGP se non ha prima una policy (filtro) configurata**. Nel laboratorio, per semplicità, viene disabilitato questo controllo globale con `no bgp ebgp-requires-policy`. Inoltre, il comando non inietta nulla nel kernel Linux e controlla sempre che la rete annunciata esista già localmente (salvo usare `no bgp network import-check`).

POLICY BGP – Filtraggio degli annunci

Un router BGP si trova in mezzo a una Policy Engine, prima di accettare una rotta in ingresso o prima di inviarla in uscita, la filtra.

- **Prefix-List:** blocca o permette i prefissi IP in ingresso o in uscita (es. "Accetta 195.11.14.0/24, rifiuta 200.1.1.0/24"). Le regole sono lette in base a un numero sequenziale (seq 5, seq 10).
Se un prefisso non fa match su nessuna riga, c'è un deny implicito alla fine: viene bloccato.
- **AS-Path Access-List:** filtra le rotte guardando i numeri degli AS attraversati usando le "Regular Expressions" (RegExp).

ATTRIBUTI BGP E IL PROCESSO DECISIONALE (Best-Path)

Ogni rotta trasporta degli attributi che ne definiscono il valore.

Gli attributi principali, nell'ordine esatto in cui un router Cisco o FRR li valuta per scegliere la rotta migliore, sono:

- **Weight** (Cisco-proprietario): valore locale al router per preferire un'uscita specifica.
- **Local Preference:** attributo discrezionale ben noto. Viene condiviso solo all'interno dell'AS (tramite iBGP). Permette agli amministratori di una azienda di decidere globalmente quale sia l'uscita preferita verso Internet.
Valore di default è 100, vince il valore più alto.
- **Originate:** si preferiscono le rotte generate localmente.
- **AS-Path Length:** numero di AS attraversati. Attributo importante per prevenire i loop.
Vince il percorso più corto, spesso i provider fanno AS-Path Prepending, ovvero aggiungono il proprio numero AS più volte per far sembrare il percorso artificialmente più lungo e disincentivare il traffico.
- **Origin code:** i (imparato da IGP), e (EGP) o ? (Incomplete). Vince la i.
- **MED** (Multi-Exit Discriminator - Metric): attributo opzionale non transitivo. Se due collegamenti arrivano allo stesso identico AS vicino (multi-homed), l'AS mittente può impostare un MED su un link. L'AS ricevente guarderà il MED per capire su quale link il mittente preferisce ricevere il traffico di ritorno. Valore di default 0, vince il valore più basso.
- **Manipolazione tramite Route-Maps.**

Per manipolare questi attributi si usano le route-map. Sono simili agli statement if-then:

- *match ip address myAccessList* (se l'annuncio corrisponde all'IP dell'ACL).
- *set metric 5 e set local-preference 25* (modifica i suoi attributi BGP)

Queste mappe vengono applicate sugli annunci in ingresso o in uscita rispetto a un vicino.

PROBLEMA DEL NEXTHOP E RECURSIVE LOOKUP

L'attributo Next-Hop dice al router dove inviare i pacchetti. C'è una regola principale: **in iBGP, l'indirizzo nexthop NON cambia** quando la rotta viene propagata.

Questo causa un problema: se un router riceve una rotta iBGP che punta all'IP di un provider esterno, ma non ha una rotta IGP per raggiungere quell'IP, la rotta BGP diventerà inutilizzabile perchè fallirà il "Recursive Lookup", spesso si risolve tramite il comando *next-hop-self*.

STUB NETWORKS E DEFAULT ROUTE

Una Stub Network è un AS cliente connesso a un solo ISP con un solo collegamento.

L'ISP non ha bisogno di connettere tutta la topologia interna del cliente, e al cliente non serve l'intera tabella BGP di Internet.

La soluzione è fare in modo che l'ISP invii al cliente una **Default Route** (0.0.0.0/0)

- Si può fare usando il comando BGP network 0.0.0.0/0 in cima alla gerarchia dell'ISP.
- Un metodo che si preferisce è applicare al vicino il comando default-originate. Questo fa apparire la rotta come se fosse generata dinamicamente e la inietta nel BGP del cliente senza toccare la tabella kernel locale del provider.

MULTI-HOMED STUB NETWORKS

Se un cliente ha due collegamenti di backup verso lo stesso provider, entra in gioco la vera progettazione delle policy di routing (Traffic Engineering). Non potendo usare rotte statiche (non saprebbero gestire i failover) si usa il BGP.

- **Policy di Backup Attivo/Passivo:** il cliente vuole che il Link 1 sia primario e il Link 2 intervenga solo se il primo si rompe.
- Per il **Traffico Inbound** (come il provider ci spedisce i pacchetti), il cliente manda un MED (metrica) più alto al router di backup (set metric 10). Dato che vince la metrica minore, il provider manderà i dati sul Link primario (che di default ha metric 0).
- Per il **Traffico Outbound** (come noi mandiamo pacchetti su Internet), il router del cliente modificherà gli annunci in ingresso ricevuti dal link di backup abbassandogli la local-preference a 90 (set local-preference 90). Di default, il link primario ha 100 e quindi vincerà la selezione del Best-Path.

LOAD SHARING TRA PROVIDER DIVERSI

Cosa succede se il cliente è multi-homed tra due provider diversi (es. ISP1 e ISP2) e vuole condividere equamente il carico (Load Sharing)?

Il meccanismo più comune per il traffico in ingresso (Inbound Loadsharing) sfrutta il concetto del **Longest Prefix Match** e non gli attributi BGP.

- Il cliente annuncia l'intero blocco grosso (200.1.0.0/16) ad entrambi gli ISP per sicurezza (se un link cade, l'altro copre tutto).
- Per forzare il bilanciamento, spacca l'indirizzo a metà in due /17 (più specifici).
- Annuncia 200.1.0.0/17 verso l'ISP1 e 200.1.128.0/17 verso l'ISP2. Poiché i prefissi /17 sono più specifici di un /16, i router globali di Internet instraderanno metà del traffico forzatamente verso l'ISP1 e metà verso l'ISP2. Se uno dei due link cade, la rotta /16 dichiarata all'inizio funzionerà da "catch-all" (rete di sicurezza) per ripristinare le comunicazioni totali.

ARCHITETTURA DEI MODERNI DATA CENTER (Clos / Fat-Tree)

BGP è diventato il protocollo standard anche all'interno delle moderne architetture Data Center.

Per far fronte al traffico enorme generato dalle applicazioni cloud e micro-servizi (traffico tra server e server) e non solo dal traffico da/verso Internet, i Data Center abbandonano le vecchie topologie gerarchiche tradizionali a favore delle reti Clos e Fat-Tree.

- Questa topologia usa **strati di switch**:
 - **Leaf**, collegati al server.
 - **Spine**, collegati ai Leaf.
 - **ToF** (Top of Fabric), per scalare ulteriormente raggruppando l'infrastruttura in blocchi chiamati PoD (Point of Delivery)
- Il vantaggio dei Fat-Tree è che creano una maglia in cui esistono sempre **molteplici percorsi di lunghezza uguale** (Multi-Path) tra un server e l'altro. Ciò garantisce una banda molto elevata e una enorme resistenza ai guasti.

ECMP (Equal-Cost Multi-Path) e BGP

Il routing in questi Data Center deve saper gestire più percorsi allo stesso tempo.

- Invece di usare i classici IGP (OSPF), i grandi Data Center usano **BGP**.
- BGP genera molto meno rumore di flooding rispetto a OSPF in reti così dense.
- BGP supporta nativamente l'**ECMP**, un meccanismo in cui, se ci sono più rotte con la stessa metrica verso una destinazione, il traffico viene bilanciato in modo uniforme su tutte le rotte valide sfruttando l'hashing a livello di kernel.

ASSEGNAZIONE DEGLI AS E BGP MULTI-PATH RELAX

In un Data Center strutturato a Fat-Tree non si usano numeri di Autonomus System (ASN) pubblici, ma AS privati (da 64512 in poi).

Spesso viene seguita la seguente convenzione:

- Ogni switch Leaf ha il proprio AS unico.
- Gli switch Spine di un PoD condividono lo stesso AS.
- I ToF condividono lo stesso AS a seconda del "piano" a cui appartengono.

Per far funzionare l'ECMP in questo scenario così segmentato, BGP ha bisogno di un "aiutino". Di default, per fare bilanciamento di carico tra due rotte, BGP richiede che la lunghezza dell'AS-PATH sia identica, ma anche che specifici numeri di AS attraversati siano identici.

Nei data center si abilita quindi la funzione `bgp bestpath as-path multipath-relax`: questa funzione ignora il controllo stringente sugli AS number esatti e verifica solo che la lunghezza del percorso (il numero di salti) sia la stessa.

MULTI-PATH TRACEROUTE

Il comando *traceroute* restituisce risultati confusi o errati se viene utilizzato ECMP.

Si utilizzano strumenti specializzati come **Paris-Traceroute** e **Dublin-Traceroute** che generano pacchetti appositi per mappare correttamente tutti i rami e i percorsi multipli che i pacchetti attraversano nella topologia Fat-Tree.

OTTIMIZZAZIONE ESTREMA – I TIMER IN DC

I timer di default sono lenti, in un Data Center BGP viene configurato per reagire ai guasti istantaneamente:

- advertisement-interval impostato a 0 secondi (default 30s): per inoltrare istantaneamente gli annunci.
- keepalive e hold timer ridotti rispettivamente a 3 e 9 secondi (default 60s/180s) per dichiarare un nodo "morto" velocemente.
- connect timer abbassato a 10 secondi (default 60s) per riavviare i peering rotti più velocemente.

LIVELLO TRASPORTO – A COSA SERVE?

Il livello trasporto si trova a metà strada tra il livello Applicativo e il livello Rete (IP).

Se l'IP si occupa già del passaggio dei pacchetti tra un dispositivo e l'altro perchè aggiungere un altro livello?

Problema principale: **Multiplexazione e Demultiplexazione**

- Un computer non esegue una singola app, ma molti processi contemporaneamente.
- Quando un pacchetto arriva al computer (via Internet), il livello IP sa solo che è arrivato alla macchina giusta, non sa a quale applicazione consegnarlo.
- Il livello trasporto aggiunge una etichetta fondamentale chiamata **Port Number**.

MECCANISMO:

- **Multiplexing** (in uscita): il livello trasporto prende i messaggi dalle varie app, aggiunge l'header con le porte (sorgente e destinazione) creando un Segmento, poi passa a IP.
- **Demultiplexing** (in entrata): leggendo il numero di porta di destinazione del Segmento ricevuto, il livello trasporto sa esattamente a quale processo consegnare il pacchetto.

Per gestire lo smistamento si utilizzano delle strutte software: le Socket.

- In **UDP**, un socket è identificato in modo univoco da due parametri (IP locale, Porta locale).
- In **TCP**, per via della sua natura orientata alla connessione, un socket richiede quattro parametri (IP locale, Porta locale, IP remoto, Porta remota).

AFFIDABILITA' SU UNA RETE AFFIDABILE

Il secondo grande compito del livello di trasporto (specialmente per il protocollo TCP) è creare l'illusione di una rete perfetta, nonostante Internet sia imperfetta. Sulla rete i pacchetti possono perdersi, arrivare in disordine, arrivare corrotti o congestionare i router.

PROCEDIMENTO PER LA COSTRUZIONE DI UN PROTOCOLLO (TEORICO)

per il trasferimento dei dati affidabile (RDT – Reliable Data Transfer)

1. Come gestire i dati corrotti?

Se i bit si invertono durante il viaggio, serve un modo per accorgersene.

- **Checksum:** il mittente calcola un valore matematico basato sui dati e lo inserisce nell'header. Il destinatario ricalcola questo valore: se non coincide, il pacchetto è corrotto.
- **ACK e NACK:** il ricevitore invia un feedback. Manda un "ACK" (Acknowledge) se il pacchetto è intatto, o un "NACK" (Negative Acknowledge) se è corrotto. Se il mittente riceve un NACK, ritrasmette i dati.

2. Corruzione dell'ACK:

Il mittente potrebbe ricevere una conferma incomprensibile e non sapere cosa fare.

- **Numero di Sequenza (SEQ):** Si etichetta ogni pacchetto con un numero progressivo. Se il mittente riceve un ACK corrotto, per sicurezza ritrasmette il pacchetto. Il ricevitore, vedendo arrivare un pacchetto con un SEQ che ha già ricevuto, sa che è un duplicato e lo scarta in sicurezza, rimandando l'ACK. Questo permette anche di eliminare i NACK: se il ricevitore manda un ACK per il pacchetto "0" quando si aspettava l' "1", equivale a dire implicitamente "rimandami l'1".

3. Come gestire la perdita totale dei dati?

Se un pacchetto (o il suo ACK) si perde completamente, la comunicazione si bloccherebbe per sempre.

- **Timer e Timeout:** il mittente avvia un timer (cronometro) quando spedisce un pacchetto. Se l'ACK non arriva prima dello scadere del timer (timeout), il mittente presume che il pacchetto o l'ACK siano andati persi e ritrasmette il dato.

PIPELINING: OTTIMIZZARE LA VELOCITA'

Il protocollo che aspetta l'ACK prima di inviare il pacchetto successivo si chiama "Stop-and-Wait". Sebbene affidabile, è lentissimo: la maggior parte del tempo il mittente non fa nulla, aspettando il segnale di ritorno.

Per sfruttare la banda, si usa il **Pipelining**: il mittente può inviare un gruppo di N pacchetti (chiamato Finestra Scorrevole o Sliding Window) consecutivamente, prima ancora di ricevere le conferme.

Come si comportano i protocolli in pipelining se si perde un pacchetto in mezzo al gruppo? Ci sono due strategie principali:

• Go-Back-N:

- **Come funziona:** gli ACK sono cumulativi. Se il ricevitore manda "ACK 10", significa "ho ricevuto tutto correttamente fino al pacchetto 10".
- **In caso di errore:** se arriva il pacchetto 1, 2, ma il 3 si perde, il ricevitore non accetta e scarta i pacchetti successivi (4,5,6) anche se sono corretti, continuando a rispondere "ACK 2".
- **Ritrasmissione:** al timeout, il mittente deve ritrasmettere il pacchetto perso (3) e tutti quelli successivi (4,5,6).

- **Selective Repeat:**
 - **Come funziona:** gli **ACK sono selettivi e indipendenti** per ogni singolo pacchetto. Il ricevitore deve avere un buffer (una memoria) per conservare i pacchetti che arrivano fuori ordine.
 - **In caso di errore:** se si perde il pacchetto 3, il ricevitore accetta e mette in memoria il 4, 5, 6, inviando ACK individuali per ciascuno.
 - **Ritrasmissione:** al timeout, il mittente ritrasmette solo il pacchetto mancante (il 3).

I FONDAMENTALI DEL TCP

Il TCP è un protocollo complesso che si assume l'incarico di risolvere quasi tutti i problemi di una rete inaffidabile. I suoi **elementi fondamentali** sono:

- **Checksums:** per rilevare la corruzione dei dati.
- **ACKs (Acknowledge):** per segnalare la ricezione avvenuta con successo. In TCP gli ACK sono cumulativi (indicano il byte successivo atteso).
- **SEQs (Sequence Numbers):** per ordinare i segmenti. A differenza di protocolli teorici che numerano a pacchetti (1, 2, 3...), TCP numera i byte di dati inviati.
- **Timeouts e Ritrasmissioni:** per recuperare i dati persi.

Oltre all'affidabilità di base, TCP implementa funzioni cruciali come **l'impostazione della connessione (Three-way Handshake con i flag SYN e ACK)**, il **Flow Control** (per non sommergere il ricevitore) e il **Congestion Control** (per non sommergere la rete).

CONTROLLO DELLA CONGESTIONE

L'obiettivo del controllo di congestione è inviare i dati il più velocemente possibile senza intasare i router di Internet. A differenza del Flow Control (in cui il ricevitore dice esplicitamente quanto spazio ha), nel controllo di congestione il mittente deve indovinare la capacità della rete calcolando una **Congestion Window (cwnd)**.

Il TCP usa un meccanismo di **"Self-Clocking"**:

- **Arrivo di un ACK:** significa che la rete è libera, aumenta la finestra.
- **Mancato arrivo (Timeout o NACK impliciti):** significa che la rete è congestionata, riduce la finestra.

LE FASI DEL TCP BASE (TCP RENO/TAHOE)

1. **Slow Start** (Aumento Esponenziale): all'avvio, la finestra è di 1 MSS (Maximum Segment Size). Per ogni ACK ricevuto, la finestra aumenta di 1 MSS. L'effetto netto è che la finestra raddoppia a ogni RTT (Round Trip Time). È una fase molto aggressiva per trovare rapidamente il limite della rete.
2. **Congestion Avoidance** (Aumento Lineare): quando la finestra raggiunge una certa soglia chiamata ssthresh (Slow Start Threshold), TCP capisce che la rete potrebbe presto congestionarsi. Passa a un aumento cauto: cresce solo di 1 MSS per ogni RTT.

COME SI ACCORGE DEGLI ERRORI E COME REAGISCE?

TCP ha due modi per accorgersi di aver perso pacchetti:

- **Timeout** (Danno grave): l'intero gruppo di pacchetti è andato perso.
 - Reazione (Tahoe/Reno): taglia la ssthresh a metà, resetta la finestra a 1 MSS e ricomincia da zero con la Slow Start.
- **3 Duplicate ACKs** (Danno lieve): il ricevitore continua a mandare lo stesso ACK perché gli manca un buco in mezzo a pacchetti che gli stanno comunque arrivando.
 - Reazione (Reno/Fast Recovery): taglia la ssthresh a metà, riduce la finestra a quel valore (più 3 MSS) e riprende con l'Aumento Lineare (evitando il doloroso riavvio da 1 MSS).

TCP CUBIC

Le vecchie versioni di TCP (come Reno) funzionano male su reti moderne molto veloci e a lunga distanza (reti con alto Bandwidth-Delay Product). Salire linearmente impiega troppo tempo per riempire un "tubo" molto capiente.

Per questo nasce TCP CUBIC, oggi standard in molti sistemi operativi (come Linux e Android).

La funzione cubica: Invece di usare linee rette e aumenti legati all'RTT, CUBIC definisce la crescita della finestra con una curva matematica cubica in funzione del tempo trascorso dall'ultima congestione. Questo lo rende indipendente dalla latenza (RTT) della rete.

La curva si divide in tre fasi:

- **Concave Region** (Fast Recovery): dopo un errore, la finestra è tagliata al 70% (non al 50% come Reno). Cresce rapidamente, ma poi la crescita decelera man mano che si avvicina al punto Wmax (il limite in cui era avvenuto l'errore precedente).
- **Plateau Region** (Cautious Probing): vicino a Wmax, la curva si appiattisce. CUBIC diventa cauto per capire se la rete è stabile ed evitare di causare subito un nuovo drop.
- **Convex Region** (Aggressive Probing): se non avvengono perdite superato Wmax, la curva s'impenna prepotentemente per trovare nuovi limiti di banda.

Inoltre, CUBIC include una modalità **"TCP-Friendly"**: se la sua crescita risultasse più lenta del vecchio TCP Reno (specialmente su reti povere), si adatta al comportamento di Reno per non essere ingiustamente penalizzato.

IL PROTOCOLLO QUIC

Protocollo di trasporto utilizzato da Google, **diventato la base di HTTP/3**.

Perché utilizzare un nuovo protocollo?

Modificare o aggiornare il TCP è difficilissimo perché è radicato profondamente nei kernel dei sistemi operativi di tutti i dispositivi del mondo. QUIC è stato costruito sopra l'UDP nello "user space". Questo significa che vive dentro i browser e le applicazioni, potendo essere aggiornato facilmente.

VANTAGGI DI QUIC

- **Velocità di setup:** TCP richiede molto tempo (vari RTT) per aprire la connessione e stabilire la crittografia TLS.
QUIC fonde queste fasi, potendo iniziare a trasmettere dati subito (0-RTT) su server già conosciuti.
- **Flussi Indipendenti** (Multiplexing senza blocco): in TCP, se si perde un pacchetto, tutto il flusso si ferma aspettando la ritrasmissione (Head-of-line blocking).
In QUIC, scaricando più file (es. immagini di un sito), se si perde il pezzo di un'immagine, le altre continuano a essere scaricate normalmente.
- **Sicurezza Integrata:** la crittografia TLS 1.3 è nativa e obbligatoria in QUIC; non esiste connessione in chiaro.
- **Connection Migration:** se passi dal Wi-Fi alla rete 4G, il tuo IP cambia. Il TCP fa cadere la connessione. QUIC usa un ID di connessione sganciato dall'IP, mantenendo il download o il video in streaming attivo e fluido pur cambiando rete.

CONTROLLO DELLA CONGESTIONE E FAIRNESS

- **L'obiettivo del Controllo di Congestione** (Collasso da Congestione): se le sorgenti di traffico inviassero dati alla loro velocità massima senza alcun feedback dalla rete, si verificherebbe un "collasso da congestione".
In questo scenario, le risorse di rete (come la banda dei link) vengono consumate per trasportare pacchetti che verranno inevitabilmente scartati più avanti lungo il percorso. Il risultato è che all'aumentare del carico offerto, il throughput utile della rete precipita verso lo zero (come dimostrato matematicamente nel modello di rete ad anello).
- **Efficienza vs Equità** (Fairness): **ottimizzare la rete solo per ottenere il massimo throughput totale ("Efficienza Paretiana") è pericoloso, perché potrebbe portare ad azzerare completamente la banda di alcune connessioni per favorirne altre (grossa iniquità).**
Serve un compromesso tra efficienza globale ed equità tra gli utenti.
- **I MODELLI DI FAIRNESS**
 - **Max-Min Fairness:** l'obiettivo è massimizzare la velocità dei flussi più lenti. Un'allocazione è "max-min fair" se non è possibile aumentare la banda di una sorgente senza dover diminuire quella di una sorgente che ha già una velocità inferiore o uguale. Si calcola tramite l'algoritmo del "Water-filling" (riempimento progressivo dell'acqua) e si basa sul concetto che ogni sorgente deve avere un "bottleneck link" (collegamento collo di bottiglia) che la limita.
 - **Proportional Fairness** (Equità Proporzionale): massimizza la somma dei logaritmi delle velocità delle sorgenti. A differenza del Max-Min, questo modello penalizza leggermente i flussi che attraversano molti router o usano molte risorse di rete, garantendo una maggiore efficienza complessiva (come dimostrato nello scenario del "parking lot").
 - **Utility Fairness:** un approccio generalizzato che mira a massimizzare una funzione di "utilità" per le sorgenti, sottraendo i "costi" della rete. Viene dimostrato matematicamente che la Max-Min Fairness è un caso limite della Utility Fairness.

- **L'Algoritmo AIMD (Additive Increase, Multiplicative Decrease):**

Il meccanismo per far convergere in modo distribuito e asincrono i router verso l'equità è l'AIMD. Analizzando i vettori geometrici, l'autore dimostra che per raggiungere un punto di equità stabile:

- La riduzione in caso di congestione deve essere moltiplicativa (es. dividere la velocità per 2).
 - L'aumento in assenza di congestione deve avere una componente additiva (es. aggiungere 1 pacchetto alla volta).
- **Analisi Matematica (ODE):** utilizzando Equazioni Differenziali Ordinarie (ODE), viene dimostrato che l'AIMD con code FIFO ha un "difetto" intrinseco: crea un pregiudizio negativo (bias) contro le connessioni che hanno un Round Trip Time (RTT) elevato.

INTERNET E TCP

Il TCP utilizza due variabili principali: cwnd (congestion window) e twnd (target window o ssthresh).

Gli Algoritmi di Congestione del TCP (Tahoe/Reno):

- **Slow Start:** all'avvio, la cwnd parte da 1 e raddoppia a ogni RTT (crescita esponenziale) finché non raggiunge la soglia twnd.
- **Congestion Avoidance:** superata la soglia, il TCP entra nella fase di "aumento additivo" (cresce linearmente di 1 MSS per RTT) per sondare la rete con cautela.
- **Fast Recovery e Reazione alle perdite:** se il TCP rileva una perdita tramite Timeout (danno grave), crolla in Slow Start; se la rileva tramite 3 ACK duplicati (danno lieve), dimezza la finestra ed entra in Fast Recovery.

EVOLUZIONI MODERNE DEL CONTROLLO DELLA CONGESTIONE

Il TCP è limitato sulle moderne reti (veloci e distanti, Long Fat Networks)

- **TCP CUBIC:** Sostituisce l'aumento lineare con una funzione matematica cubica. Questa curva cresce lentamente vicino al punto dove si era verificata l'ultima congestione (fase concava e di plateau) e poi accelera aggressivamente se non ci sono pacchetti persi (fase convessa), rendendo la crescita della finestra indipendente dall'RTT.
- **Data Center TCP (DCTCP):** nei data center le latenze sono microscopiche e ci sono picchi di traffico enormi. Il DCTCP modifica la diminuzione moltiplicativa: invece di tagliare sempre la finestra del 50%, stima la probabilità di congestione e applica un taglio proporzionale. Ciò garantisce finestra e throughput più alti e stabili.

GESTIONE ATTIVA DELLE CODE (AQM) E ECN

- **Bufferbloat e RED:** usare la caduta del pacchetto (tail drop) a coda piena causa il "Bufferbloat" (alta latenza perché i buffer restano sempre saturi). La soluzione è l'AQM (Active Queue Management) tramite l'algoritmo RED (Random Early Detection). RED scarta probabilisticamente i pacchetti prima che il buffer si riempia, basando il calcolo su una media esponenziale ponderata della lunghezza della coda (avg).
- **ECN** (Explicit Congestion Notification): invece di scartare fisicamente il pacchetto causando ritrasmissioni, un router ECN lo "marca" modificando un bit nell'header IP. Il ricevitore avvisa il mittente (tramite il flag TCP ECE), permettendogli di ridurre la finestra di congestione senza subire alcuna vera perdita di dati.

ISOLAMENTO DEL TRAFFICO

- **Applicazioni "TCP-Friendly":** applicazioni video o audio che usano UDP dovrebbero implementare algoritmi (come TFRC) basati sulla Loss-Throughput formula, per comportarsi in modo educato ("TCP-Friendly") e non togliere banda ai flussi TCP legittimi.
- **Class Based Queuing:** in reti chiuse, il traffico UDP aggressivo viene arginato usando code basate su classi e schedulatori pesati (Weighted Fair Queuing) per garantire isolamento e banda minima a tutti.